TASK 4: MINI PYTHON PROJECT

STUDENT MANAGEMENT SYSTEM — 40 MARKS

1. Project Title

Student Management System

2. Objective

To develop a simple, user-friendly Python application that can add, view, search, and delete student records. The project demonstrates programming concepts covered in Weeks 1–4, including functions, control statements, data structures, file handling, exception handling, and Python libraries.

3. Problem Statement

Managing student details manually can be time-consuming and may lead to errors or misplaced records. This project provides a console-based system to organize student information and save records in a file for future use.

4. Features

- Add a student with name, age, and marks.
- Generate a student ID automatically.
- View all student records.
- Search for a student by ID.
- Delete a student record by ID.
- Save student records in a JSON file so they remain available after restarting.
- Handle invalid numeric input and common file errors.

5. Concepts Used

Requirement	Implementation
Variables and data types	name (string), age (integer), marks (float)
Input and output	input(), print()
Conditional statements	if, elif, else
Loops	while and for loops
Functions	add_student(), view_students(), search_student(), delete_student()
Data structures	List and dictionary
File handling	open(), json.load(), json.dump()
Exception handling	try/except for ValueError, JSONDecodeError, and OSError
Python libraries	json, os, random

6. Source Code

```
import json
import os
import random

FILE_NAME = "students.json"

# Load saved student records
def load_students():
    if os.path.exists(FILE_NAME):
        try:
            with open(FILE_NAME, "r") as file:
                return json.load(file)
        except (json.JSONDecodeError, OSError):
            print("Error reading file. Starting with empty records.")
    return []

# Save records to a JSON file
def save_students(students):
    try:
        with open(FILE_NAME, "w") as file:
            json.dump(students, file, indent=4)
    except OSError:
        print("Error: Unable to save student records.")

# Add a student
def add_student(students):
    try:
        name = input("Enter student name: ").strip()
        if not name:
            print("Name cannot be empty.")
            return

        age = int(input("Enter age: "))
        marks = float(input("Enter marks (0-100): "))

        if age <= 0 or not 0 <= marks <= 100:
            print("Invalid age or marks.")
            return

        student = {
            "id": random.randint(1000, 9999),
            "name": name,
            "age": age,
            "marks": marks
        }
        students.append(student)
        save_students(students)
        print("Student added successfully!")
        print("Student ID:", student["id"])

    except ValueError:
        print("Error: Enter valid numeric values.")

# View all students
def view_students(students):
    if not students:
        print("No student records found.")
        return

    print("\n--- Student Records ---")
    for student in students:
        print("ID:", student["id"])
```

```
        print("Name:", student["name"])
        print("Age:", student["age"])
        print("Marks:", student["marks"])
        print("-----------------------")

# Search for a student by ID
def search_student(students):
    student_id = input("Enter student ID: ").strip()
    for student in students:
        if str(student["id"]) == student_id:
            print("Student Found!")
            print(student)
            return
    print("Student not found.")

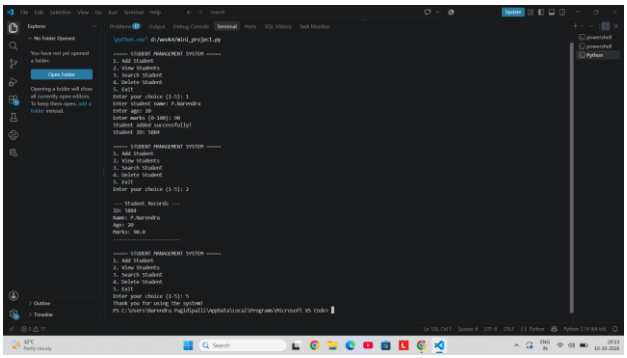
# Delete a student by ID
def delete_student(students):
    student_id = input("Enter student ID to delete: ").strip()
    for student in students:
        if str(student["id"]) == student_id:
            students.remove(student)
            save_students(students)
            print("Student deleted successfully!")
            return
    print("Student not found.")

# Main menu
def main():
    students = load_students()
    while True:
        print("\n===== STUDENT MANAGEMENT SYSTEM =====")
        print("1. Add Student")
        print("2. View Students")
        print("3. Search Student")
        print("4. Delete Student")
        print("5. Exit")

        choice = input("Enter your choice (1-5): ").strip()
        if choice == "1":
            add_student(students)
        elif choice == "2":
            view_students(students)
        elif choice == "3":
            search_student(students)
        elif choice == "4":
            delete_student(students)
        elif choice == "5":
            print("Thank you for using the system!")
            break
        else:
            print("Invalid choice. Please select 1 to 5.")

if __name__ == "__main__":
    main()
```

7. Output



Note: The student ID shown is an example. Your program will generate a random ID.

8. Project Explanation

When the program starts, load_students() reads saved records from students.json. The menu repeats with a while loop until the user chooses Exit. Each menu option calls a separate function. Student records are dictionaries stored inside a list. The random library generates IDs, and the json library saves and loads records. try/except blocks handle invalid numeric input and common file errors.

9. Challenges Faced

- Handling non-numeric age or marks without crashing the program.
- Validating that age is positive and marks are between 0 and 100.
- Saving records so they remain available after restarting the program.
- Displaying a clear message when a student ID cannot be found.

10. Conclusion

The Student Management System demonstrates how Python can be used to build a practical record-management application. It combines variables, input/output, conditions, loops, functions, data structures, file handling, exception handling, and Python libraries.